BÀI TẬP LÝ THUYẾT – LẬP KẾ HOẠCH VÀ ĐỒ THỊ KẾ HOẠCH TRONG AI

Họ và tên	Phan Thanh Tùng
MSSV / Lớp	3123410417

MỤC TIÊU HỌC TẬP

- Mô tả bài toán lập kế hoạch cổ điển bằng trạng thái, mục tiêu, toán tử và các tập Precondition/Add/Delete.
- Xây dựng planning graph theo GraphPlan với các mức trạng thái S_i, mức hành động A_i và các hành động No-op.
- Nhận diện và giải thích các quan hệ mutex: Inconsistent Effects, Interference và Competing Needs.
- Phân tích điều kiện dừng của GraphPlan và trích xuất kế hoạch từ planning graph.
- Vận dụng GraphPlan vào các bài toán Crane, Valet Parking và môi trường planning rời rạc.

Hướng dẫn: Trả lời bằng tiếng Việt, trình bày rõ các bước suy luận. Với câu có planning graph, có thể vẽ trực tiếp lên hình hoặc bổ sung bảng. No-op được tính là một hành động khi xét mutex.

Quy ước: NoOp(p) là no-op của mệnh đề p; hiệu ứng ¬p trong Add List nghĩa là p bị phủ định; hai mệnh đề p và ¬p luôn mutex với nhau.

BÀI 1. KHÁI NIỆM CƠ BẢN TRONG GRAPHPLAN

a) Interference (xung đột do can thiệp)

Định nghĩa: hai hành động a, b ở cùng mức A_i interfere khi hiệu ứng xóa (phủ định) của một hành động phá hủy một tiền điều kiện của hành động kia:

$$\text{Neg(Eff(a))} \cap \text{Pre(b)} \neq \varnothing \quad \text{hoặc} \quad \text{Neg(Eff(b))} \cap \text{Pre(a)} \neq \varnothing$$

Ý nghĩa: nếu cho chạy cùng lúc thì hành động bị phá tiền điều kiện có thể không còn thực hiện được, kết quả phụ thuộc vào thứ tự. Vì vậy hai nút hành động được nối bằng cạnh mutex và không được chọn cùng nhau trong một mức khi trích xuất kế hoạch.

b) Inconsistent Effects (hiệu ứng không nhất quán)

Định nghĩa: hai hành động a, b có hiệu ứng không nhất quán khi một hành động phủ định hiệu ứng của hành động kia (một bên thêm p, bên kia xóa p):

$$\text{Neg(Eff(a))} \cap \text{Add(b)} \neq \varnothing \quad \text{hoặc} \quad \text{Neg(Eff(b))} \cap \text{Add(a)} \neq \varnothing$$

Ý nghĩa: chạy cùng lúc thì trạng thái kết quả không xác định. Cặp hành động bị đánh dấu mutex, và hệ quả là các mệnh đề chỉ có thể đạt được nhờ cặp hành động này sẽ mutex ở mức trạng thái sau.

c) Competing Needs (tiền điều kiện cạnh tranh)

Định nghĩa: hai hành động a, b ở mức A_i là competing needs khi tồn tại $p \in \text{Pre(a)}$, $q \in \text{Pre(b)}$ mà p và q đã mutex ở mức trạng thái S_i.

Ý nghĩa: p, q không thể cùng đúng ở S_i nên a và b không thể cùng được thực hiện. Khác hai loại trên, loại này không do tác động trực tiếp của hai hành động mà do thông tin mutex ở mức trước truyền sang.

Từ ba loại mutex hành động, hai mệnh đề ở S_{i+1} là mutex khi chúng đối lập nhau hoặc mọi cặp hành động có thể đạt được chúng đều mutex (inconsistent support).

d) Các cách tìm kế hoạch trong lập kế hoạch cổ điển

- **Tìm kiếm không gian trạng thái:** tiến (progression) từ trạng thái đầu hoặc lùi (regression) từ mục tiêu, dùng BFS, DFS, A* kèm heuristic.
- **Lập kế hoạch thứ tự từng phần (POP):** tìm trong không gian các kế hoạch từng phần, thêm hành động, ràng buộc thứ tự và liên kết nhân quả, xử lý các mối đe dọa.
- **GraphPlan:** dựng planning graph rồi trích xuất kế hoạch bằng tìm kiếm ngược, dùng mutex để cắt tỉa.
- **Quy về SAT hoặc CSP (SATPlan):** mã hóa bài toán với độ dài kế hoạch k thành công thức mệnh đề rồi dùng bộ giải SAT.
- **Tìm kiếm heuristic từ bài toán nới lỏng:** h_{add}, h_{max}, FF kết hợp A* hoặc hill-climbing.
- **Lập kế hoạch phân cấp (HTN):** phân rã nhiệm vụ cấp cao thành các hành động cụ thể.

e) Các giả định được nới lỏng khi dùng heuristic dựa trên GraphPlan

Heuristic lấy từ planning graph ($h_{levelsum}$, h_{max}, kế hoạch nới lỏng của FF) giải một bài toán đã được nới lỏng:

- Bỏ qua delete list (hiệu ứng âm): mệnh đề đã đúng thì không bao giờ bị phá, số mệnh đề chỉ tăng qua các mức.
- Bỏ qua hoặc chỉ xét một phần quan hệ mutex: coi các mục tiêu và tiền điều kiện là độc lập, đạt đồng thời được.
- Cho phép nhiều hành động cùng thực hiện trong một mức, không còn ràng buộc mỗi bước một hành động.

Vì sao hữu ích hơn tìm kiếm ngây thơ:

- Bài toán nới lỏng giải được trong thời gian đa thức, còn số trạng thái của bài toán gốc tăng theo hàm mũ.
- Mức mà một mệnh đề xuất hiện lần đầu cho ước lượng số bước còn lại để đạt nó, nên A* hay hill-climbing được dẫn hướng về mục tiêu thay vì duyệt mù như BFS, DFS.

BÀI 2. LẬP KẾ HOẠCH KÝ HIỆU – BÀI TOÁN CẦN CẨU

Một cần cẩu, một kiện hàng K, một xe tải T. Ban đầu K ở bên trái, T ở bên phải, cần cẩu ở bên trái (theo đề). Mục tiêu: đưa K lên xe tải và cuối cùng cần cẩu quay về bên trái. Hai vị trí: L (trái), R (phải).

a) Đối tượng, predicate, trạng thái đầu và mục tiêu

Predicate	Ý nghĩa
CraneAt(x)	Cần cẩu đang ở vị trí x, x ∈ {L, R}
ContainerAt(x)	Kiện hàng đang nằm trên mặt đất ở vị trí x
TruckAt(x)	Xe tải đang ở vị trí x
Holding	Cần cẩu đang nhấc kiện hàng
Free	Cần cẩu đang rảnh, không nhấc gì
OnTruck	Kiện hàng đã nằm trên xe tải

$$S_0 = \{\text{CraneAt(L)}, \text{ContainerAt(L)}, \text{TruckAt(R)}, \text{Free}\}$$

$$\text{Mục tiêu} = \{\text{OnTruck}, \text{CraneAt(L)}\}$$

Xe tải không di chuyển nên TruckAt(R) luôn đúng.

b) Các toán tử hành động

Toán tử	Preconditions	Add List	Delete List
Move(x, y)	CraneAt(x)	CraneAt(y)	CraneAt(x)
Pick(x)	CraneAt(x) ContainerAt(x) Free	Holding	ContainerAt(x) Free
Load(x)	CraneAt(x) TruckAt(x) Holding	OnTruck Free	Holding

Move dùng được khi có hoặc không mang kiện hàng: nếu Holding đang đúng thì nó giữ nguyên sau khi di chuyển, tức kiện hàng đi cùng cần cẩu. Load là cách duy nhất đưa kiện hàng lên xe tải.

c) Planning graph cho 3 mức hành động đầu tiên (S_0 đến S_3)

Mức S_0: CraneAt(L), ContainerAt(L), TruckAt(R), Free.

Mức A_0: áp dụng được Move(L,R) và Pick(L), cộng 4 no-op. Move(R,L), Pick(R), Load không áp dụng được vì thiếu tiền điều kiện.

Hành động	Cạnh tiền điều kiện (từ S_0)	Cạnh hiệu ứng (tới S_1)
Move(L,R)	CraneAt(L)	Thêm CraneAt(R); xóa CraneAt(L)
Pick(L)	CraneAt(L), ContainerAt(L), Free	Thêm Holding; xóa ContainerAt(L), Free
NoOp(p)	$p \in S_0$	$p \in S_1$

Mức S_1: CraneAt(L), ContainerAt(L), TruckAt(R), Free, CraneAt(R), Holding.

Mutex ở S_1: {CraneAt(L), CraneAt(R)}, {CraneAt(R), Holding}, {Holding, ContainerAt(L)}, {Holding, Free}.

Mức A_1: Move(L,R), Move(R,L), Pick(L) và 6 no-op. Load(R) chưa có vì CraneAt(R) và Holding mutex ở S_1; Pick(R) không có vì không có ContainerAt(R).

Hành động	Cạnh tiền điều kiện (từ S_1)	Cạnh hiệu ứng (tới S_2)
Move(L,R)	CraneAt(L)	Thêm CraneAt(R); xóa CraneAt(L)
Move(R,L)	CraneAt(R)	Thêm CraneAt(L); xóa CraneAt(R)
Pick(L)	CraneAt(L), ContainerAt(L), Free	Thêm Holding; xóa ContainerAt(L), Free
NoOp(p)	$p \in S_1$	$p \in S_2$

Mức S_2: vẫn 6 mệnh đề như S_1. Mutex ở S_2: {CraneAt(L), CraneAt(R)}, {ContainerAt(L), Holding}, {Free, Holding}.

Cặp {CraneAt(R), Holding} hết mutex vì đạt được bằng Move(L,R) cùng NoOp(Holding), hai hành động này không xung đột.

Mức A_2: Load(R) áp dụng được (CraneAt(R), TruckAt(R), Holding đôi một không mutex), cùng Move(L,R), Move(R,L), Pick(L) và 6 no-op.

Hành động	Cạnh tiền điều kiện (từ S_2)	Cạnh hiệu ứng (tới S_3)
Load(R)	CraneAt(R), TruckAt(R), Holding	Thêm OnTruck, Free; xóa Holding
Move(L,R), Move(R,L), Pick(L)	Như ở A_1	Như ở A_1
NoOp(p)	$p \in S_2$	$p \in S_3$

Mức S_3: 6 mệnh đề cũ và OnTruck (do Load(R)).

Nhận xét: OnTruck và CraneAt(L) đều có mặt ở S_3 nhưng mutex với nhau.

- OnTruck chỉ do Load(R) đạt được.
- Load(R) mutex với Move(R,L): Interference, vì Move xóa CraneAt(R) là tiền điều kiện của Load.
- Load(R) mutex với NoOp(CraneAt(L)): Competing Needs, vì CraneAt(R) và CraneAt(L) mutex ở S_2.

Kết luận: chưa thể trích xuất kế hoạch ở S_3, cần mở rộng đến S_4 ứng với kế hoạch Pick(L), Move(L,R), Load(R), Move(R,L).

d) Hai mức đầu tiên (A_0, A_1) và các cặp hành động mutex

Hai mức đầu là S_0, A_0, S_1, A_1, S_2 ở câu c. Các cặp không nêu thì không mutex.

Mức A_0:

STT	Cặp hành động	Loại mutex và lý do
1	Move(L,R) và Pick(L)	Interference: Move xóa CraneAt(L), là tiền điều kiện của Pick.
2	Move(L,R) và NoOp(CraneAt(L))	Inconsistent Effects: Move xóa CraneAt(L), no-op thêm CraneAt(L).
3	Pick(L) và NoOp(ContainerAt(L))	Inconsistent Effects: Pick xóa ContainerAt(L), no-op thêm lại.
4	Pick(L) và NoOp(Free)	Inconsistent Effects: Pick xóa Free, no-op thêm lại.

Mức A_1:

STT	Cặp hành động	Loại mutex và lý do
1	Move(L,R) và Move(R,L)	Inconsistent Effects: Move(L,R) thêm CraneAt(R) còn Move(R,L) xóa CraneAt(R), và ngược lại với CraneAt(L).
2	Move(L,R) và Pick(L)	Interference: Move xóa CraneAt(L), tiền điều kiện của Pick.
3	Move(R,L) và Pick(L)	Competing Needs: CraneAt(R) và CraneAt(L) mutex ở S_1.
4	Move(L,R) và NoOp(CraneAt(L))	Inconsistent Effects: Move xóa CraneAt(L), no-op thêm CraneAt(L).
5	Move(R,L) và NoOp(CraneAt(R))	Inconsistent Effects: Move(R,L) xóa CraneAt(R), no-op thêm CraneAt(R).
6	Move(L,R) và NoOp(CraneAt(R))	Competing Needs: CraneAt(L), CraneAt(R) mutex ở S_1.
7	Move(R,L) và NoOp(CraneAt(L))	Competing Needs: CraneAt(R), CraneAt(L) mutex ở S_1.
8	Move(R,L) và NoOp(Holding)	Competing Needs: CraneAt(R), Holding mutex ở S_1.
9	Pick(L) và NoOp(ContainerAt(L))	Inconsistent Effects: Pick xóa ContainerAt(L).
10	Pick(L) và NoOp(Free)	Inconsistent Effects: Pick xóa Free.
11	Pick(L) và NoOp(CraneAt(R))	Competing Needs: CraneAt(L) (của Pick), CraneAt(R) mutex ở S_1.

STT	Cặp hành động	Loại mutex và lý do
12	Pick(L) và NoOp(Holding)	Competing Needs: ContainerAt(L), Free (của Pick) mutex với Holding ở S_1.
13	NoOp(CraneAt(L)) và NoOp(CraneAt(R))	Competing Needs: CraneAt(L), CraneAt(R) mutex ở S_1.
14	NoOp(CraneAt(R)) và NoOp(Holding)	Competing Needs: CraneAt(R), Holding mutex ở S_1.
15	NoOp(Holding) và NoOp(ContainerAt(L))	Competing Needs: Holding, ContainerAt(L) mutex ở S_1.
16	NoOp(Holding) và NoOp(Free)	Competing Needs: Holding, Free mutex ở S_1.

BÀI 3. QUAN HỆ MUTEX TRONG PLANNING GRAPH

Pinky đi lấy thức ăn. Trạng thái đầu: Rich ∧ At(Home). Mục tiêu: Full. Viết tắt CC = ChuckyCheese.

Hành động	Preconditions	Add List	Delete List
Move(A,B)	At(A)	At(B)	At(A)
Buy(Cheese)	At(CC), Rich	Has(Cheese), ¬Rich	
Gamble	At(CC), Rich	¬Rich	Rich
ATM	At(CC), ¬Rich	Rich	¬Rich
Eat(Cheese)	Has(Cheese)	Full	Has(Cheese)

Các mức của planning graph (theo hình trong đề):

Mức	Nội dung
S_0	At(Home), Rich
A_0	Move(Home,CC), NoOp(At(Home)), NoOp(Rich)
S_1	At(Home), At(CC), Rich
A_1	Move(Home,CC), Move(CC,Home), Buy(Cheese), Gamble, và các no-op
S_2	At(Home), At(CC), Has(Cheese), ¬Rich, Rich
A_2	Move(Home,CC), Move(CC,Home), Buy(Cheese), Eat(Cheese), Gamble, ATM, và các no-op
S_3	At(Home), At(CC), Has(Cheese), ¬Rich, Rich, Full

a) Mức A_0: cặp mutex do Inconsistent Effects

Cặp: {Move(Home,CC), NoOp(At(Home))}.

Delete(Move(Home,CC)) = {At(Home)}.

Add(NoOp(At(Home))) = {At(Home)}.

Một hành động xóa At(Home), hành động kia thêm At(Home): hai hiệu ứng mâu thuẫn.

Kết luận: mutex do Inconsistent Effects.

b) Mức A_1: cặp mutex do Interference

Cặp: {Move(CC,Home), Buy(Cheese)}.

Delete(Move(CC,Home)) = {At(CC)}.

Pre(Buy(Cheese)) = {At(CC), Rich} chứa At(CC).

Add(Move(CC,Home)) = {At(Home)} không mâu thuẫn với Add(Buy(Cheese)) = {Has(Cheese), ¬Rich}.

Kết luận: mutex do Interference (hiệu ứng của Move phá tiền điều kiện của Buy). Cặp khác cũng thỏa: {Gamble, Buy(Cheese)}, vì Gamble xóa Rich là tiền điều kiện của Buy.

c) Mức A_2: cặp mutex do Competing Needs

Cặp: {Gamble, ATM}.

Pre(Gamble) = {At(CC), Rich}.

Pre(ATM) = {At(CC), ¬Rich}.

Rich và ¬Rich mutex ở S_2 vì đối lập nhau.

Kết luận: mutex do Competing Needs. Ví dụ khác: {Eat(Cheese), Gamble}: Has(Cheese) chỉ đến từ Buy(Cheese) (hiệu ứng ¬Rich của nó phủ định Rich) còn Rich ở S_2 chỉ đến từ NoOp(Rich), nên Has(Cheese) và Rich mutex.

d) Giải thích vì sao mỗi cặp thỏa loại mutex tương ứng

- Câu a: Move(Home,CC) và no-op có hiệu ứng đối lập về At(Home) (xóa và thêm), không thể cùng thực hiện mà trạng thái kết quả vẫn xác định: Inconsistent Effects.
- Câu b: nếu Move(CC,Home) chạy trước thì Pinky đã rời Chucky Cheese và không còn At(CC) để mua phô mai. Hành động này phá tiền điều kiện của hành động kia nên thứ tự quyết định kết quả: Interference.
- Câu c: không hành động nào trực tiếp phá hiệu ứng của hành động kia, nhưng Rich và ¬Rich không thể cùng đúng ở S_2 nên không có trạng thái nào cho phép cả hai hành động chạy: Competing Needs.

BÀI 4. CLASSICAL PLANNING VÀ GRAPHPLAN – VALET PARKING

Hai predicate: ClearBehind(car), ParkedBehind(car1, car2). Hai toán tử:

Thành phần	ParkBehind(car1, car2)	ParkInNewRow(car)
Preconditions	ClearBehind(car1) ClearBehind(car2) ParkedBehind(car1, place)	ClearBehind(car) ParkedBehind(car, place) ¬ParkedBehind(car, curb)
Add List	ParkedBehind(car1, car2) ClearBehind(place) ¬ParkedBehind(car1, place) ¬ClearBehind(car2)	ParkedBehind(car, curb) ClearBehind(place) ¬ParkedBehind(car, place)
Delete List	ClearBehind(car2) ParkedBehind(car1, place)	ParkedBehind(car, place) ¬ParkedBehind(car, curb)

Trạng thái S_0 trong hình:

{ParkedBehind(B,A), ParkedBehind(A,Curb), ¬ParkedBehind(B,Curb), ParkedBehind(C,Curb), ClearBehind(B), ClearBehind(C)}

a) Tên hành động ở các ô 1 đến 4 của mức A_0

Xe không bị chắn phía sau (ClearBehind) ở S_0 là B và C. ParkBehind cần cả hai xe Clear nên chỉ có (B, C) và (C, B). ParkInNewRow(car) cần car Clear và ¬ParkedBehind(car, Curb), chỉ B thỏa.

Ô	Tập tiền điều kiện (cạnh vào từ S_0)	Hành động
1	ParkedBehind(B,A), ClearBehind(B), ClearBehind(C)	ParkBehind(B, C), place = A
2	ParkedBehind(B,A), ¬ParkedBehind(B,Curb), ClearBehind(B)	ParkInNewRow(B), place = A
3	ParkedBehind(C,Curb), ClearBehind(B), ClearBehind(C)	ParkBehind(C, B), place = Curb

Ô	Tập tiền điều kiện (cạnh vào từ S_0)	Hành động
4	ParkedBehind(C,Curb), ClearBehind(C)	N/A

Ô 4 là N/A: hành động duy nhất dạng này là ParkInNewRow(C) nhưng cần ¬ParkedBehind(C,Curb). Xe C đang đỗ ở Curb nên mệnh đề này sai ở S_0 (S_0 chỉ có ¬ParkedBehind(B,Curb)).

b) Các predicate ở S_1 nhận add-edge từ từng hành động

Hành động	Predicate ở S_1 nhận add-edge
Ô 1: ParkBehind(B, C)	ParkedBehind(B,C), ClearBehind(A), ¬ParkedBehind(B,A), ¬ClearBehind(C)
Ô 2: ParkInNewRow(B)	ParkedBehind(B,Curb), ClearBehind(A), ¬ParkedBehind(B,A)
Ô 3: ParkBehind(C, B)	ParkedBehind(C,B), ClearBehind(Curb), ¬ParkedBehind(C,Curb), ¬ClearBehind(B)
Ô 4	N/A

Các add-edge lấy từ Add List của toán tử, thay car1, car2, place theo từng hành động.

c) Hai action node mutex do Inconsistent Effects (được dùng no-op)

Cặp: {ParkBehind(B,C) (ô 1), NoOp(ParkedBehind(B,A))}.

Delete(ô 1) chứa ParkedBehind(B,A).

Add(NoOp(ParkedBehind(B,A))) = {ParkedBehind(B,A)}.

Kết luận: hai hiệu ứng mâu thuẫn, mutex do Inconsistent Effects. Tương tự {ParkInNewRow(B) (ô 2), NoOp(¬ParkedBehind(B,Curb))}, vì ô 2 xóa ¬ParkedBehind(B,Curb).

d) Hai action node mutex do Interference

Cặp: {ParkBehind(B,C) (ô 1), ParkBehind(C,B) (ô 3)}.

Ô 1 xóa ClearBehind(C), là tiền điều kiện của ô 3.

Ô 3 xóa ClearBehind(B), là tiền điều kiện của ô 1.

Kết luận: hai hành động phá tiền điều kiện của nhau nên không thể chạy cùng lúc: Interference.

e) GraphPlan "leveled off" khi nào?

☐ A. Tất cả các hành động có thể đã được khám phá.
☐ B. Không còn tập literal khác rỗng nào không có liên kết mutex.
☒ **C. Hai mức liên tiếp của planning graph giống hệt nhau.**
☐ D. Mức trạng thái cuối cùng chứa một trạng thái mục tiêu.

Đáp án C. Khi hai mức trạng thái liên tiếp có cùng tập mệnh đề và cùng các cặp mutex thì mở rộng thêm không sinh thông tin mới. Nếu lúc đó mục tiêu vẫn chưa thỏa thì kết luận không có kế hoạch.

BÀI 5. XÂY DỰNG GRAPHPLAN VÀ TRÍCH XUẤT KẾ HOẠCH

Trạng thái đầu: A. Mục tiêu: C ∧ D ∧ E. Sáu toán tử:

	op1	op2	op3	op4	op5	op6
Precondition	A	B	A	A	A	A
Add	B	C, D, E	C	D	E	E, ¬A
Delete						

a) Chạy GraphPlan

Mức S_0 = {A}.

Mức A_0: chỉ toán tử có tiền điều kiện A áp dụng được: op1, op3, op4, op5, op6. op2 chưa dùng được vì B $\notin S_0$. Thêm NoOp(A) để duy trì A.

Hành động	Cạnh vào (từ S_0)	Cạnh Add (tới S_1)	Ghi chú
op1	A	B	
op3	A	C	
op4	A	D	
op5	A	E	
op6	A	E, ¬A	Hiệu ứng ¬A phủ định A
NoOp(A)	A	A	Duy trì A sang S_1

Mức S_1 = {A, B, C, D, E, ¬A}.

Mutex ở A_0: op6 mutex với từng hành động còn lại có tiền điều kiện A (op1, op3, op4, op5, NoOp(A)), tổng cộng 5 cặp.

Lý do: hiệu ứng ¬A của op6 phủ định tiền điều kiện A của các hành động đó (Interference).

Với NoOp(A) còn có Inconsistent Effects vì no-op thêm A.

Mutex ở S_1: {A, ¬A}, {¬A, B}, {¬A, C}, {¬A, D}.

{¬A, E} không mutex vì E đạt được bằng chính op6, cùng hành động với ¬A.

Các cặp như {B, C}, {C, D}, {D, E} không mutex vì op1, op3, op4, op5 đôi một không xung đột.

Điều kiện dừng: C, D, E $\in S_1$ và đôi một không mutex nên chuyển sang trích xuất kế hoạch. Trích xuất thành công ngay nên không cần mở rộng đến S_2.

b) Kế hoạch GraphPlan trả về

Tìm ngược từ tập mục tiêu {C, D, E} ở S_1:

C chỉ có op3 đạt được; D chỉ có op4; E do op5 hoặc op6.

op6 mutex với op3 và op4 nên phải chọn op5.

op3, op4, op5 đôi một không mutex và đều chỉ cần A (có ở S_0).

Kết luận: kế hoạch {op3, op4, op5} nằm trong cùng một mức A_0, gồm 3 thao tác (có thể thực hiện theo thứ tự bất kỳ, ví dụ op3, op4, op5).

c) Kế hoạch trên có tối ưu về số lượng thao tác không?

☐ Có ☒ **Không**

Có kế hoạch ngắn hơn: op1 rồi op2.

Từ {A}, op1 thêm B.

op2 (cần B) thêm C, D, E.

Kế hoạch này chỉ có 2 thao tác. GraphPlan tối ưu về số mức (bước thời gian khi cho phép hành động song song), không tối ưu về tổng số hành động: {op3, op4, op5} có 1 mức nhưng 3 thao tác, còn op1, op2 có 2 mức nhưng 2 thao tác.

Nếu mỗi thời điểm chỉ thực hiện một toán tử thì phải coi mọi hành động thật cùng mức là mutex (planning graph tuần tự). Khi đó mục tiêu chỉ thỏa ở S_2 và kế hoạch tìm được chính là op1, op2.

d) Các cặp mutex ở A_0 và S_1

- Cặp toán tử mutex ở A_0 (xét theo planning graph chuẩn, tính cả no-op): (op6, op1), (op6, op3), (op6, op4), (op6, op5), (op6, NoOp(A)).
- Cặp proposition mutex ở S_1: (A, ¬A), (¬A, B), (¬A, C), (¬A, D).

BÀI 6. TỔNG HỢP VÀ PHÂN TÍCH

a) Tìm kiếm trực tiếp trong không gian trạng thái so với GraphPlan

- Tìm kiếm trạng thái làm việc với từng trạng thái cụ thể: có thể tới 2^n trạng thái với n mệnh đề, hệ số nhánh lớn, dễ duyệt lặp cùng một trạng thái theo nhiều thứ tự hành động độc lập.
- GraphPlan dựng đồ thị có kích thước đa thức theo số mệnh đề, số hành động và số mức, rồi chỉ tìm kiếm trên đồ thị đó, kèm mutex và các tập mục tiêu thất bại (nogood) để cắt tỉa.

Planning graph giúp giảm đáng kể không gian tìm kiếm khi:

- Có nhiều hành động độc lập thực hiện song song: tìm kiếm trạng thái phải xét mọi hoán vị, GraphPlan gộp chúng vào một mức không cần sắp thứ tự.
- Có nhiều cặp hành động hoặc mệnh đề mutex: các tổ hợp không thể xảy ra bị loại ngay khi xét.
- Có nhiều hành động không liên quan đến mục tiêu: chỉ phần đồ thị dẫn tới mục tiêu được duyệt ngược.
- Cần phát hiện bài toán vô nghiệm sớm: đồ thị leveled off mà mục tiêu vẫn thiếu hoặc mutex thì kết luận ngay không có kế hoạch.

b) Vai trò của No-op

No-op biểu diễn việc một mệnh đề được giữ nguyên sang mức sau nếu không có hành động nào xóa nó (lời giải cho frame problem trong đồ thị). Nếu bỏ toàn bộ No-op:

- Mệnh đề chỉ xuất hiện ở S_{i+1} khi có hành động vừa thêm nó, nên sự thật đã đúng từ trước mà không được thêm lại sẽ biến mất. Ví dụ ở Bài 5, A không có mặt ở S_1 và mọi hành động cần A sau đó đều không dùng được.
- Không biểu diễn được trạng thái tồn tại qua nhiều mức: hành động ở mức sau không dựa được vào điều kiện đã tạo ra từ mức trước.
- Các tính chất đơn điệu (mệnh đề chỉ tăng, mutex chỉ giảm) không còn đúng nên điều kiện leveled off mất ý nghĩa.
- GraphPlan có thể bỏ sót kế hoạch hợp lệ hoặc kết luận sai là không có kế hoạch. No-op còn dùng để biểu diễn mutex giữa hành động xóa một mệnh đề và việc giữ nguyên mệnh đề đó.

c) Vai trò của mutex giữa các literal mục tiêu

- Mục tiêu nhiều literal: chỉ có mặt cùng ở một mức S_i là chưa đủ, các literal còn phải đôi một không mutex.
- Nếu hai literal mục tiêu mutex thì không có trạng thái nào ở mức đó thỏa cả hai, nên chưa thể trích xuất kế hoạch: mở rộng đồ thị thêm một mức rồi kiểm tra lại.
- Trong tìm ngược, tập mục tiêu con chứa cặp mutex bị loại ngay, giúp cắt tỉa mạnh.
- Nếu đồ thị đã leveled off mà literal mục tiêu vẫn thiếu hoặc vẫn mutex thì kết luận thất bại, không có kế hoạch.

d) Quy trình trích xuất kế hoạch từ planning graph

Giả sử các literal mục tiêu đã cùng xuất hiện ở mức S_n và đôi một không mutex.

Bước 1: đặt G_n = các literal mục tiêu ở S_n.

Bước 2: với i = n về 1, với mỗi literal g ∈ G_i, chọn một hành động (có thể là no-op) ở A_{i-1} có cạnh thêm tới g. Nếu hành động đã chọn ở mức này đã đạt g thì dùng lại. Các hành động chọn trong cùng một mức phải đôi một không mutex.

Bước 3: lập G_{i-1} là hợp các tiền điều kiện của những hành động vừa chọn (tiền điều kiện của no-op là chính literal đó). G_{i-1} không được chứa cặp mutex.

Bước 4: nếu một mức không có cách chọn nhất quán thì quay lui, thử hành động khác cho literal trước đó, và ghi tập mục tiêu thất bại vào bảng nogood để khỏi thử lại.

Bước 5: khi tới S_0 mà G_0 nằm trong trạng thái đầu thì thành công. Kế hoạch là các hành động đã chọn ở A_0 đến A_{n-1}, bỏ các no-op; hành động cùng mức thực hiện song song hoặc theo thứ tự tùy ý.

Bước 6: nếu mọi cách chọn đều thất bại ở mức n thì mở rộng đồ thị thêm một mức rồi trích xuất lại. Nếu đồ thị đã leveled off và bảng nogood không còn thay đổi thì kết luận không có kế hoạch.